

하네스 엔지니어링 — 회사 정책을 코드로 자동 적용한다

1강 settings.json·권한 + 2강 Hooks + 3강 Skills·Slash·MCP (총 120분)

코스	Course 02 · 사내 AI 구축·운영 종합 가이드
강의	1강(60분) + 2강(60분) + 3강(60분) = 총 180분
대상	IT 담당자 + 시니어 개발자
URL	visionc.co.kr/ai-solution/academy/build-ai
비밀번호	visioncDown (대소문자 구분)
형식	사내 출강 / 워크숍

INTRO · TITLE

하네스 엔지니어링

1강 — settings.json으로 회사 정책 코드화

 3분 · 시작·환영

SAY

“반갑습니다. 사내 AI 구축·운영 가이드 6편 ‘하네스 엔지니어링’입니다. ★ 표시 본격 운영 편.”

SAY

“6편 3강 구성. 1강 ‘settings.json으로 권한·환경 표준화’. 2강 ‘Hooks로 정책 자동 적용’. 3강 ‘Skills·Slash·MCP 결합’. 180분 후 ‘우리 회사 하네스 풀스택 1페이지’.”

SAY

“6편 약속 — ‘하네스(harness)’ = ‘말의 마구’ 비유. 야생 말을 안전하게 다루는 마구. 에이전트라는 ‘강력하지만 위험한 말’을 ‘회사 정책 안에서’ 운영하는 도구. 6편이 그 마구 만들기.”

Q

도입 질문 1: “‘하네스 엔지니어링’ 들어본 분?” → 보통 없음. → ‘Anthropic·OpenAI 등 회사가 만든 용어. 1년 안 표준 용어.’

Q

도입 질문 2: “5편 ‘Claude Code 시작’ 결정한 분?” → 보통 다수. → ‘6편이 그 운영 본격. 회사 정책 코드화.’

NOTE

★ 강사 핵심 배경지식 — 하네스의 의미: 단순 ‘설정’이 아닌 ‘코드 + 정책 + 자동화 결합’. 회사 정책 매뉴얼(종이) → 코드(자동)로 변환. 직원 실수 X 시스템 차단. 사내 AI 운영의 핵심.

NOTE

강사 배경지식 — 6편 출처 (1차): (1) Anthropic Claude Code 공식 문서 docs.claude.com/claude-code. (2) settings.json 사양. (3) Hooks 가이드. (4) Skills 'Introducing Agent Skills'. (5) Anthropic 사내 'How teams use Claude Code'. 모두 1차.

NOTE

강사 배경지식 — 6편 60분 시간 배분: 0~3분 인사. 3~6분 학습목표. 6~10분 5편 복습. 10~16분 하네스 풀이. 16~24분 settings.json 구조. 24~32분 Permission Modes. 32~40분 env-hooks. 40~46분 한국 표준. 46~52분 우선순위·git 공유. 52~58분 실습. 58~60분 마무리.

TIP

강사 함정 — settings.json 문법 디테일 X: ‘구조 + 결정’만. 디테일은 사내 IT.

BRIDGE

“학습 목표 빠르게.”

OBJECTIVES

60분 후, 이 3가지를 한다

- ① settings.json 구조 (allow / ask / deny) 설명
- ② 한국 회사 권장 settings 5종 패턴 안다
- ③ ‘우리 회사 settings.json 표준 1페이지’ 완성

🕒 3분 · 학습목표

SAY

“60분 후 셋. settings.json 구조, 한국 권장 패턴, 표준 1페이지.”

SAY

“1강 핵심 메시지 — ‘settings.json은 회사 권한 매트릭스. 1회 작성 + 전사 공유 + 운영’.”

NOTE

★ 강사 핵심 배경지식 — 1강 성공 기준: 청중이 ‘우리 회사 settings.json 시작 + 핵심 권한 결정’.

NOTE

강사 배경지식 — settings.json의 ‘레벨 3가지’: (1) 사용자 (개인) — ~/.claude/settings.json. (2) 프로젝트 (팀) — .claude/settings.json. (3) Enterprise (회사 전체) — managed. 3레벨 결합.

TIP

강사 진행 팁 — 학습 목표 3분 안에.

BRIDGE

“5편 복습 + 6편 위치.”

RECAP · PART 5

5편 복습 — 에이전트 시작

■ 불변증 — 5편의 핵심

- 1강 — 에이전트 4요소 + 5패턴 + 4단계 + ROI
- 2강 — Claude Code (CLI + 권한 + Skills + Hooks + MCP)
- 3강 — 오픈 4종 (OpenCode · Cline · Aider · Roo Code)
- ★ 추천 시작 — 개발팀 + Claude Code + 시니어 1명
- ★ 6편 위치 — 5편의 ‘권한·Skills·Hooks·MCP’ 본격 디테일

🕒 4분 · 5편 복습

SAY

“5편 복습 30초씩.”

SAY

“1강 에이전트 4요소·5패턴·4단계·ROI. ‘작게 시작’.”

SAY

“2강 Claude Code. 공식 에이전트. 4가지 기능.”

SAY

“3강 오픈 4종. 자체 호스팅 옵션.”

SAY

“추천 시작 — 개발팀 + Claude Code + 시니어 1명 + Pro \$20.”

SAY

“6편 위치 — 5편의 ‘권한·Skills·Hooks·MCP’ 본격 디테일. ‘운영 코드화’.”

NOTE

★ 강사 핵심 배경지식 — 5편 vs 6편: 5편 = 도구 결정 + 도입. 6편 = 운영 표준화 + 자동화. ‘도구는 같지만 운영 디테일 다름’.

TIP

강사 진행 팁 — 4분 안에.

BRIDGE

“복습 끝. 본격적으로 — 하네스 풀이.”

WHAT IS HARNESS

하네스 = 회사 정책 + 코드 + 자동

■ 불변증 — 본질

- 정책 매뉴얼 + 직원 실수 (하네스 없이) — 회사 매뉴얼(종이) — ‘민감 데이터 외부 송신 금지’. 직원 실수로 위반. 사고. 사후 처벌.
- 코드 + 자동 차단 + 안전 (하네스 사용) — settings.json·Hooks가 ‘민감 데이터 자동 감지 + 차단’. 직원 실수 가능성 0. 사전 방지.

SAY

“1강 본문 1번 — 하네스 풀이. ‘회사 정책 + 코드 + 자동’.”

SAY

“하네스 없이 — 매뉴얼 + 직원 실수. 사고 + 사후 처벌. ‘정책 문서 100페이지’ 직원 다 읽기 X.”

SAY

“하네스 사용 — 코드 + 자동. 직원이 ‘민감 데이터 송신 시도’ → 자동 감지 + 차단 + 알림. 사전 방지.”

SAY

“원칙 — ‘인간 신뢰’가 아닌 ‘시스템 차단’. 인간은 실수 가능 — 시스템은 차단 가능. 본 강좌 9편 보안 본질.”

SAY

“하네스 구성 — (1) settings.json (권한). (2) Hooks (동작 전후). (3) Skills (도메인 지식). (4) Slash Commands (단축). 4가지 결합.”

STORY

Anthropic 사내 ‘.claude/’ 폴더 출처. Anthropic 'How Anthropic teams use Claude Code'. URL: claude.com/blog/how-anthropic-teams-use-claude-code. 사내 모든 개발자 ‘.claude/settings.json + Hooks + Skills’ 공유. ‘사내 표준 = 코드’.

STORY

한국 회사 하네스 도입 사례 (강사 대응용). 카카오·네이버 일부 개발팀 — 6개월 운영 후 ‘settings.json + 5~10 Hooks + 10~20 Skills’ 표준. 사내 코딩 가이드 + 보안 정책 + 자동 테스트 결합.

STORY

하네스 ROI (강사 대응용). 도입 전 — 매월 사고 1~2건. 도입 후 — 0건 + 효율 +30%. 1회 작성 + 영구 적용 패턴.

NOTE

★ 강사 핵심 배경지식 — 하네스 4가지 도구 위치: (1) settings.json — 무엇 할 수 있나 (정적 권한). (2) Hooks — 무엇 할 때 자동 (동적 동작). (3) Skills — 무엇 하는 방법 (지식). (4) Slash Commands — 자주 쓰는 명령. 4가지 다 결합 = 폴스택.

NOTE

강사 배경지식 — 하네스의 ‘레벨’: (1) 개인 — 자기 설정. (2) 팀 — 프로젝트 공유. (3) 회사 — Enterprise managed. 큰 회사일수록 ‘회사 레벨’ 표준.

NOTE

강사 배경지식 — 하네스 ‘없으면’ 함정: 첫 Claude Code 사용자 자주 ‘settings 그대로 사용’. 함정 — (1) 위험 명령 자동 허용. (2) 사내 정책 X. (3) 사고 시 추적 어려움. 하네스 없는 도입 = 1~3개월 후 사고.

Q

예상 청중 질문 1: ‘우리는 첫 도입. 하네스 X 시작?’ → 답: ‘기본 settings.json 권장 — 위험 명령 차단 + 사내 정책 추가. 0에서 시작 X.’

Q

예상 청중 질문 2: ‘하네스 작성 누가?’ → 답: ‘사내 시니어 + IT부. 첫 작성 1~2주. 이후 ‘업무 발견 시 추가.’’

Q

예상 청중 질문 3: ‘오픈 에이전트 하네스도?’ → 답: ‘Cline·Aider 등은 별도 권한 시스템. Claude Code의 settings.json + Hooks·Skills 가장 풍부. 본 강좌 6편 = Claude Code 기준.’

TIP

강사 함정 — 하네스 깊이 X: ‘본질 + 가치’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 본질. 1~2분 비유. 2~4분 4도구. 4~5분 한국 사례. 5~6분 청중 질문.

BRIDGE

“하네스 봤다면 — settings.json 구조.”

SETTINGS STRUCTURE

settings.json 구조 — allow · ask · deny

■ 불변층 — 핵심 3개 배열

- ★ allow — 자동 허용 (예: 'Bash(git status)')
- ★ ask — 매번 승인 요청 (예: 'Edit(**)')
- ★ deny — 절대 차단 (예: 'Bash(rm -rf *)')
- ★ 패턴 문법 — 'Tool(인자)' + 와일드카드(**)
- ★ Tools — Read·Edit·Bash·WebFetch·MCP·기타
- ★ 우선순위 — deny > ask > allow > 기본

🕒 6분 · settings 구조

SAY

“settings.json 구조. 3개 핵심 배열 — allow·ask·deny.”

SAY

“allow — 자동 허용. 'Bash(git status)' — 매번 승인 X. 빠른 작업.”

SAY

“ask — 매번 승인 요청. 'Edit(**)' — 파일 수정 매번 사용자 승인. 안전.”

SAY

“deny — 절대 차단. 'Bash(rm -rf *)' — 위험 명령 절대 X.”

SAY

“패턴 문법 — 'Tool(인자)' + 와일드카드 '**'. 'Bash(git **)' = 모든 git 명령. 'Edit(src/**)' = src 폴더 파일만.”

SAY

“Tools — Read(읽기)·Edit(수정)·Bash(명령)·WebFetch(웹)·MCP(외부)·기타. 약 10가지.”

SAY

“우선순위 — deny > ask > allow > 기본. 같은 명령에 ‘deny’와 ‘allow’ 둘 다 있으면 deny 적용.”

STORY

settings.json 공식 출처. Anthropic Claude Code docs.claude.com/claude-code/settings. JSON 형식. ‘permissions’ 키 + ‘allow·ask·deny’ 배열. 약 30가지 키 (env·hooks·model 등) 있음 — 6편 1강에서는 권한 위주.

STORY

settings.json 권장 예시 (강사 대응용). { 'permissions': { 'allow': ['Read(**)', 'Bash(git status)', 'Bash(npm test)'], 'ask': ['Edit(**)', 'Bash(git push)', 'WebFetch(*)'], 'deny': ['Bash(rm -rf *)', 'Bash(sudo *)', 'Edit(.env)', 'Edit(secrets/**)'] } }. 한국 회사 안전 시작 패턴.

STORY

Tool 종류 디테일 (강사 대응용). (1) Read — 파일 읽기. (2) Edit — 파일 수정. (3) Write — 새 파일. (4) Bash — 셸 명령. (5) WebFetch — URL 다운로드. (6) WebSearch — 웹 검색. (7) Task — 서버에이전트. (8) Skill — 스킬 실행. (9) Glob — 파일 검색. (10) Grep — 텍스트 검색. + MCP 도구.

NOTE

★ 강사 핵심 배경지식 — settings.json 작성 5단계: (1) 위험 명령 ‘deny’ 첫 작성. (2) 자주 쓰는 안전 명령 ‘allow’. (3) 나머지 ‘ask’. (4) 1주 운영 후 ‘자주 ask’ → ‘allow’ 이동. (5) 사내 표준화 + git 공유. 1~4주 안에.

NOTE

강사 배경지식 — 와일드카드 패턴 디테일: (1) ‘**’ — 모든 인자. (2) ‘src/**’ — src 폴더 안 전부. (3) ‘*.md’ — 모든 .md 파일. (4) ‘Bash(npm *)’ — npm으로 시작하는 명령. 정규식 X — 단순 glob 패턴.

NOTE

강사 배경지식 — settings.json 레벨 3가지: (1) 사용자 — ~/.claude/settings.json (개인). (2) 프로젝트 — .claude/settings.json (팀, git). (3) Enterprise — managed (회사 정책 강제). 우선순위 — Enterprise > 사용자 > 프로젝트.

Q

예상 청중 질문 1: ‘처음부터 deny 매트릭스?’ → 답: ‘권장. ‘rm·sudo·env·secrets’ 4가지 기본 deny. 1시간 작성.’

Q

예상 청중 질문 2: ‘ask 매번 답답?’ → 답: ‘초기 안전 우선. 1주 후 ‘자주 ask’ → allow 이동.’

Q

예상 청중 질문 3: ‘다른 개발자와 settings 공유?’ → 답: ‘.claude/settings.json 프로젝트 레벨 + git 커밋. 팀 동기화 자연.’

TIP

강사 함정 — JSON 문법 자세히 X: ‘구조’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 3배열. 1~3분 패턴·Tools. 3~4분 우선순위. 4~5분 작성 5단계. 5~6분 청중 질문.

BRIDGE

“구조 봤다면 — *Permission Modes.*”

PERMISSION MODES

Permission Modes — 4가지 운영 모드

■ 불변층 — 상황별 모드 전환

- default 표준 모드 — settings.json 권한 따름. 안전 + 효율 균형. 평시 운영.
- plan 계획 전용 (읽기만) — Edit·Bash·Write 모두 X. ‘이 코드 분석’ 같은 읽기만. 사고 위험 0. 첫 도입 권장.
- acceptEdits 수정 자동 허용 — Edit ask → allow 자동 전환. 빠른 작업. 단 위험 큼. 단기 사용.
- bypassPermissions 모든 권한 자동 — 모든 ask·deny 무시. ★ 절대 위험. ‘격리 환경 (sandbox)’ 외 사용 금지.

SAY

“Permission Modes — 4가지 운영 모드.”

SAY

“default — 표준. settings.json 따름. 평소 운영.”

SAY

“plan — 계획 전용. 읽기만. Edit·Bash·Write X. ‘이 코드 분석’ 같은 작업. 사고 위험 0. 첫 도입 권장.”

SAY

“acceptEdits — 수정 자동 허용. Edit ask → allow. 빠른 작업. 위험 큼. 단기 사용.”

SAY

“bypassPermissions — 모든 권한 자동. ★ 절대 위험. ‘격리 환경 (sandbox)’ 외 사용 금지.”

SAY

“전환 — 명령줄 ‘claude --permission-mode plan’ 또는 슬래시 ‘/permissions plan’ 같은.”

STORY

Permission Modes 공식. Anthropic Claude Code docs.claude.com/claude-code/permissions.
‘plan mode’는 안전 첫 시작 권장. ‘bypassPermissions’은 ‘격리 환경 (Docker·Cloud Sandbox)에서만’ 명시.

STORY

한국 회사 모드 사용 패턴 (강사 대응용). (1) 신입 개발자 — plan 1주. (2) 일반 운영 — default. (3) 빠른 작업 — acceptEdits 30분~1시간. (4) bypass — Anthropic Managed Agents sandbox 같은 격리만. 안전 첫 사용.

STORY

‘plan mode’의 의미 (강사 대응용). 사용자 첫 도입 시 ‘에이전트가 무엇 할지 보기’만. 자율 동작 X. 1~2주 plan 사용 후 ‘안전 확인’ → default 전환. 시스템 학습 안전.

NOTE

★ 강사 핵심 배경지식 — 4모드 사용 흐름: (1) 첫 1주 — plan. (2) 1~4주 — default + ask 다수. (3) 1~3개월 — default + allow 확대. (4) 3개월+ — 본격. bypass는 절대 격리 외 X.

NOTE

강사 배경지식 — bypassPermissions의 ‘격리 환경’: Anthropic Managed Agents self-hosted sandbox 또는 Docker 격리. ‘사내 메인 시스템 X 연결’. 사고 시 ‘격리 환경 안’만 영향. 4편 3강 MCP 격리와 동일 원칙.

Q

예상 청중 질문 1: ‘첫 도입 plan 1주?’ → 답: ‘권장. ‘에이전트 무엇 할 수 있나’ 관찰. 자율 동작 X. 안전.’

Q

예상 청중 질문 2: ‘acceptEdits 안전?’ → 답: ‘ask 대신 자동 Edit. 빠른 작업에 의미. 단 ‘긴 자율 작업’엔 사고 위험. 단기 사용.’

Q

예상 청중 질문 3: ‘bypass 무엇?’ → 답: ‘본격 자율 격리 환경. ‘Anthropic Managed Agents self-hosted sandbox’ 같은. 평소 사내 X.’

TIP

강사 함정 — bypass 깊이 X: ‘격리 외 X’ 메시지.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 4모드. 1~3분 각 1분. 3~4분 흐름. 4~5분 bypass 위험. 5~6분 청중 질문.

BRIDGE

“모드 봤다면 — `env-hooks.additionalDirectories` 같은 다른 키.”

ENV·HOOKS·DIRS

env · hooks · 기타 키

■ 불변층 — settings.json 추가 키

- ★ env — 환경변수 (API 키·플래그 등)
- ★ hooks — Hooks 정의 (2강 자세히)
- ★ additionalDirectories — 추가 작업 폴더
- ★ model — 사용 모델 (claude-3-5-sonnet 등)
- ★ statusLine — 상태 줄 커스터마이징
- ★ 약 30가지 키 — Claude Code 공식 docs 참조

🕒 5분 · env·hooks 키

SAY

“settings.json은 permissions 외에도 다양 키. 5가지 흔한 키 + 약 30가지.”

SAY

“env — 환경변수. ‘ANTHROPIC_API_KEY’, ‘DISABLE_AUTOUPDATER=1’ 같은. 모든 작업에 적용.”

SAY

“hooks — 2강 자세히. 동작 전후 자동 함수.”

SAY

“additionalDirectories — 기본 폴더 외 추가. ‘../shared’ 같은 외부 폴더.”

SAY

“model — 사용 모델. ‘claude-3-5-sonnet-20241022’ 같은. 작업별 다른 모델.”

SAY

“statusLine — Claude Code 상태 줄. ‘우리 회사 로고·현재 모델·세션 시간’ 등.”

SAY

“총 약 30가지 키. ‘우리는 권한 + env + hooks 3가지’ 우선.”

STORY

settings.json 전체 키 (강사 대응용). docs.claude.com/claude-code/settings. (1) permissions. (2) env. (3) hooks. (4) model. (5) statusLine. (6) additionalDirectories. (7) outputStyle. (8) includeCoAuthoredBy. (9) defaultMode. (10) cleanupPeriodDays. + 20가지+. 본 강좌 핵심 3~5개만.

STORY

Anthropic 사내 statusLine 사례 (강사 대응용). 일부 회사 ‘로고·세션·모델·온도’ 표시. 직원 즉시 인식. 한국 회사 ‘회사 로고 + 부서’ 추가 권장.

NOTE

★ 강사 핵심 배경지식 — settings.json 핵심 4키: (1) permissions — 권한 (1강). (2) env — 환경변수. (3) hooks — 자동 동작 (2강). (4) statusLine — 사내 브랜딩. 4가지 결합.

NOTE

강사 배경지식 — env의 ‘회사 표준’ 가치: ‘DISABLE_AUTOUPDATER=1’ (회사 통제 업데이트) + ‘ANTHROPIC_BASE_URL=사내 프록시’ (보안). 회사 표준 + 보안 통일.

Q

예상 청중 질문 1: ‘env 직접 작성?’ → 답: ‘settings.json env 키 + 값. ANTHROPIC_API_KEY 같은. 단 ‘API 키 보안’ — git에 X. 환경변수 별도 또는 secrets.’

Q

예상 청중 질문 2: ‘model 키 의미?’ → 답: ‘기본 모델 지정. ‘claude-3-5-sonnet’ 또는 ‘claude-opus-4-1’. 작업별 다른 모델 사용 가능.’

Q

예상 청중 질문 3: ‘30가지 다 외워야?’ → 답: ‘X. 4가지 핵심만. 나머지 ‘필요 시 docs 검색’.’

TIP

강사 함정 — 30키 디테일 X: ‘4핵심’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 5가지 키. 1~3분 각 30초. 3~4분 핵심 4. 4~5분 청중 질문.

BRIDGE

“키 봤다면 — 한국 표준 5종.”

KOREA STANDARDS

한국 회사 settings.json 5종 패턴

■ 가변층 — 2026.6 권장 패턴

- 첫 도입 1주 (최소 안전) — deny 4종 (rm/sudo/.env/secrets) + ask 전부 + allow X. 안전 + 매번 승인. 1주 운영 후 확장.
- 1~3개월 안정 (일반 운영) — deny 5종 + ask 일부 (Edit·Bash 일부) + allow 자주 (Read·git status·npm test). 효율 + 안전 균형.

🕒 6분 · 5종 패턴

SAY

“한국 회사 settings.json 5종 패턴. 단계별.”

SAY

“1 최소 안전 (첫 도입 1주). deny 4종 + ask 전부 + allow X. 매번 승인. 안전.”

SAY

“2 일반 운영 (1~3개월). deny 5종 + ask 일부 + allow 자주. 균형.”

SAY

“추가 3종 — (3) 본격 자율 (3개월+), (4) 자체 호스팅 + 보안 강박, (5) Enterprise 통제 — 슬라이드 외 청중 질문 시.”

SAY

“권장 — 1 → 2 → 3 점진 확장. ‘완벽 settings’ X — 운영 후 보완.”

STORY

한국 회사 settings.json 패턴 5종 (강사 대응용 추정). (1) 최소 안전 — 첫 도입 1주. (2) 일반 운영 — 1~6개월. (3) 본격 자율 — 6개월+. (4) 자체 호스팅 + 보안 강박 — 사내 LLM. (5) Enterprise — 회사 강제 통제. 90% 한국 회사 1~3.

STORY

Anthropic 사내 settings 패턴 (강사 대응용). ‘Skills + Hooks + MCP 통합 + 부서별 settings’. 일반 직원·부서·관리자 3레벨. 사내 표준화. 한국 회사 ‘부서별 settings’ 패턴 참고.

NOTE

★ 강사 핵심 배경지식 — 5종 패턴 진화: 1 (1주) → 2 (1~3개월) → 3 (3~6개월) → 4·5 (6개월+ 회사 따라). 단계별 진화 자연. 처음부터 5 X.

NOTE

강사 배경지식 — ‘자체 호스팅 + 보안 강박’ 패턴: env에 사내 프록시 URL. allow에 사내 도구만. deny에 외부 fetch. + Ollama·vLLM 사내 LLM. 본 강좌 9편 자세히.

Q

예상 청중 질문 1: ‘1주 후 2 자동 전환?’ → 답: ‘수동 검토. 1주 운영 + 사고 X + 효율 측정 → 2 전환. 시니어 결정.’

Q

예상 청중 질문 2: ‘5 Enterprise 도입?’ → 답: ‘큰 회사 + 사내 강제 통제. 직원 개인 settings 무효화. 본 강좌 9·11편.’

Q

예상 청중 질문 3: ‘다른 회사 settings 참고?’ → 답: ‘GitHub ‘.claude/settings.json’ 검색 + Anthropic 공식 예시. 단 ‘우리 회사 정책에 맞게 조정’ 필수.’

TIP

강사 함정 — 5종 디테일 깊이 X: ‘단계 진화’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 5종. 1~3분 1·2. 3~4분 3~5. 4~5분 진화. 5~6분 청중 질문.

BRIDGE

“5종 봤다면 — 우선순위 + git 공유.”

PRIORITY·GIT

우선순위 + git 공유

■ 불변층 — 3레벨 결합

- ★ 우선순위 — Enterprise > 사용자 > 프로젝트 > 기본
- ★ 사용자 — ~/.claude/settings.json (개인)
- ★ 프로젝트 — .claude/settings.json (팀, git 커밋)
- ★ Enterprise — managed (회사 강제, 직원 X 변경)
- ★ git 공유 — 프로젝트 레벨 + CLAUDE.md 결합
- ★ secrets·API 키 — git X. env 별도

🕒 5분 · 우선순위·git

SAY

“settings.json 우선순위 + git 공유.”

SAY

“우선순위 — Enterprise > 사용자 > 프로젝트 > 기본. ‘회사 강제’ > ‘개인 설정’ > ‘팀 공유’ > ‘기본값’.”

SAY

“사용자 레벨 — ~/.claude/settings.json. 개인. ‘내 단축키·취향’.”

SAY

“프로젝트 레벨 — .claude/settings.json. 팀 공유. git 커밋. ‘이 프로젝트 표준’.”

SAY

“Enterprise 레벨 — managed. 회사 강제 통제. 직원 X 변경. ‘회사 보안 정책’.”

SAY

“git 공유 — 프로젝트 레벨 + CLAUDE.md 결합. 팀 신규 멤버 자동 동일 환경.”

SAY

“★ secrets·API 키 — git X. .gitignore에 .env. 환경변수 별도 관리.”

STORY

settings.json 우선순위 디테일 (강사 대응용). docs.claude.com/claude-code/settings — ‘Enterprise managed settings override all’. 회사 강제 통제. 직원이 ‘deny 우회 X’ 보장.

STORY

git 공유 패턴 (강사 대응용). 한국 회사 — .claude/ 폴더에 (1) settings.json. (2) commands/. (3) skills/. (4) CLAUDE.md. 4가지 git 공유. 신규 개발자 ‘프로젝트 clone’ 시 즉시 같은 환경.

NOTE

★ 강사 핵심 배경지식 — git 공유의 4가지 가치: (1) 신규 멤버 즉시 표준. (2) 정책 1회 작성 + 영구. (3) git history 정책 변경 추적. (4) PR 리뷰로 정책 검증. ‘회사 정책 = 코드 + git 흐름’.

NOTE

강사 배경지식 — .gitignore 패턴: secrets·키·개인 settings — git X. ‘.env, .env.local, .claude/settings.local.json’ 같은. 첫 도입 시 .gitignore 작성 필수.

Q

예상 청중 질문 1: ‘우리 회사 Enterprise 설정 가능?’ → 답: ‘Claude Enterprise 구독 + Anthropic 협의. 큰 회사 본격 도입.’

Q

예상 청중 질문 2: ‘개인 + 프로젝트 충돌?’ → 답: ‘우선순위 따름. 사용자 > 프로젝트. 단 ‘프로젝트 deny = 개인 allow 이김’. 실제로는 ‘개인이 프로젝트 덮어쓰기’ 어렵게 설계.’

Q

예상 청중 질문 3: ‘직원 1명 자기 settings 빼고 쓰면?’ → 답: ‘Enterprise 적용 시 X. 사용자·프로젝트만이면 가능 — 단 ‘프로젝트 정책 무시 시 sanctions’.’

TIP

강사 함정 — Enterprise 디테일 X: ‘3레벨’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 우선순위. 1~3분 3레벨. 3~4분 git 공유. 4~5분 secrets·청중 질문.

BRIDGE

“공유 봤다면 — 보안 패턴 + 사고 방지.”

SECURITY

보안 패턴 + 사고 방지

■ 불변증 — settings.json 보안 핵심

- ★ secrets·API 키 — git 절대 X (.gitignore 필수)
- ★ deny 5종 — rm/sudo/.env/secrets/외부 fetch
- ★ ask 강제 — Edit(**) + Bash(중요 명령)
- ★ Enterprise 강제 — 회사 정책 직원 X 우회
- ★ 정기 검토 — 분기 1회 settings 감사
- ★ 사고 시 — 로그 + 권한 즉시 강화

🕒 5분 · 보안

SAY

“settings.json 보안 패턴 6가지.”

SAY

“1 secrets·API 키 git 절대 X. .gitignore + 환경변수 또는 1Password·Bitwarden 같은 secrets manager.”

SAY

“2 deny 5종 기본 — rm·sudo·env·secrets·외부 fetch. 첫 settings 5종 기본.”

SAY

“3 ask 강제 — Edit(**) + Bash(중요 명령). 자동 X. 사용자 매번 인지.”

SAY

“4 Enterprise 강제 — 직원 X 우회. 회사 보안 핵심.”

SAY

“5 정기 검토 — 분기 1회 settings 감사. ‘위험 명령 추가됐나’, ‘우회 X’.”

SAY

“6 사고 시 — 로그 + 권한 즉시 강화. 같은 사고 방지.”

STORY

한국 회사 settings 사고 사례 (강사 대응용). 일부 회사 — 직원이 자기 settings에 ‘allow Bash(**)’ 추가 → 위험 명령 자동 → 사고. 대응 — Enterprise 강제 + 정기 감사. 본 강좌 9편 자세히.

STORY

secrets manager 도구 (강사 대응용). 1Password·Bitwarden·AWS Secrets Manager·HashiCorp Vault. 한국 회사 — 1Password Enterprise 또는 사내 Vault. settings.json에는 ‘env에 placeholder’ + 실행 시 secrets manager에서 가져오기.

NOTE

★ 강사 핵심 배경지식 — settings.json 보안 5단계 운영: (1) 도입 — deny 5종 기본. (2) 1개월 — Enterprise 검토. (3) 분기 — 감사 + 정책 업데이트. (4) 사고 시 — 즉시 강화. (5) 연간 — 전체 정책 재검토. 5 단계 사이클.

NOTE

강사 배경지식 — 권한 매트릭스 ‘Tool × 역할’ 패턴: 신입 — Read·git status 자동. 시니어 — Edit·Bash 자동. 시니어+ — bypass 격리. 관리자 — Enterprise. 4역할 × N Tools 매트릭스.

Q

예상 청중 질문 1: ‘우리 회사 secrets manager X?’ → 답: ‘1Password Business \$8/인 또는 사내 Vault. 한국 회사 대부분 도입 중. 본 강좌 9편 자세히.’

Q

예상 청중 질문 2: ‘Enterprise 무엇 강제?’ → 답: ‘deny·env·sandbox 강제. 직원 X 우회. Claude Enterprise 구독.’

Q

예상 청중 질문 3: ‘사고 시 누가 대응?’ → 답: ‘사내 보안팀 + IT부. 사전 ‘사고 대응 플레이북’ 작성. 본 강좌 9·10편.’

TIP

강사 함정 — 보안 디테일 깊이 X: ‘5단계 사이클’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 6패턴. 1~3분 각 30초. 3~4분 5단계. 4~5분 청중 질문.

BRIDGE

“보안 봤다면 — 권한 매트릭스.”

MATRIX

권한 매트릭스 — Tool × 역할

■ 불변층 — 사내 표준

- ★ 역할 — 신입 / 시니어 / 시니어+ / 관리자
- ★ Tools — Read·Edit·Bash·WebFetch·MCP
- ★ 신입 — Read + 기본 Bash auto, 나머지 ask
- ★ 시니어 — Edit auto + Bash 다수 auto
- ★ 시니어+ — bypass 격리 환경 가능
- ★ 관리자 — Enterprise 설정

🕒 4분 · 매트릭스

SAY

“권한 매트릭스 — Tool × 역할. 사내 표준.”

SAY

“역할 4가지 — 신입·시니어·시니어+·관리자.”

SAY

“Tools 5가지 — Read·Edit·Bash·WebFetch·MCP.”

SAY

“신입 — Read·기본 Bash 자동. Edit·고급 Bash ask. 안전.”

SAY

“시니어 — Edit·Bash 다수 자동. ask 일부. 효율.”

SAY

“시니어+ — bypass 격리 가능. 본격 자율.”

SAY

“관리자 — Enterprise 설정. 회사 정책 강제.”

STORY

권한 매트릭스 예시 (강사 대응용). | Role | Read | Edit | Bash | WebFetch | MCP | | 신입 | auto | ask | ask | ask | ask | | 시니어 | auto | auto | auto | ask | auto | | 시니어+ | auto | auto | auto | auto | auto + bypass | | 관리자 | Enterprise 강제 |. 한국 회사 권장 패턴.

NOTE

★ 강사 핵심 배경지식 — 매트릭스 실제 구현: 사용자 settings에 역할별 다른 권한. 또는 Enterprise 통제. 사내 IT가 ‘직원별 settings 자동 배포’.

NOTE

강사 배경지식 — 역할 ‘자동 승격’: 신입 1년 → 시니어 자동. ‘시니어+’는 신청 + 시니어 추천 + 관리자 승인. 본 강좌 9편.

Q

예상 청중 질문 1: ‘우리 회사 역할 적합?’ → 답: ‘4역할 표준. 회사 규모·문화에 따라 조정. 5명+ 회사 권장.’

Q

예상 청중 질문 2: ‘bypass 시니어+ 의미?’ → 답: “격리 환경에서 bypass’ — 사내 sandbox 또는 클라우드. 본격 자율. 6개월 운영 후 권장.’

Q

예상 청중 질문 3: ‘매트릭스 시간?’ → 답: ‘초기 작성 1주. 사내 표준화 1개월. 사고 시 업데이트.’

TIP

강사 함정 — 매트릭스 ‘완벽 압박’ X: ‘기본 + 조정’.

TIP

강사 진행 팁 — 4분 시간 배분: 0~1분 4역할. 1~2분 매트릭스. 2~3분 구현. 3~4분 청중 질문.

WORKSHOP 1

실습 — 우리 회사 settings.json 표준

■ 적용 (워크시트)

- Q1 · 단계 — 최소 안전(1주) / 일반(1~3개월) / 본격?
- Q2 · deny 5종 — rm·sudo·env·secrets·외부 fetch?
- Q3 · 자주 allow — git status·npm test·기타?
- Q4 · 역할 매트릭스 — 신입/시니어/시니어+/관리자?
- Q5 · git 공유 — .claude/settings.json + CLAUDE.md?
- ★ 다음 액션 — 시니어 1명 1주 안 작성 + 공유

🕒 8분 · 실습

SAY

“1강 마지막 — 실습. settings.json 표준 5칸.”

DO

조별 실습 (5분, 3~4인 1조).

SAY

“권장 시작 — 최소 안전 + deny 5종 + Read·git status allow + 신입·시니어 2역할 + git 공유. 1주 작성.”

Q

발표 요청: “단계 + 역할 수.” → 다양.

NOTE

★ 강사 핵심 배경지식 — 결정서 5칸 가이드: (1) Q1 — 첫 도입은 최소 안전. (2) Q2 — 5종 기본. (3) Q3 — git status·npm test 권장. (4) Q4 — 2역할 시작. (5) Q5 — .claude/ git 공유.

NOTE

강사 배경지식 — 다음 주 액션 3가지: (1) 시니어 1명 1주 작성. (2) 팀 PR + 리뷰. (3) git 커밋 + 사내 표준. 2주 안에.

TIP

강사 함정 — ‘완벽 settings 압박’ X.

TIP

강사 진행 팁 — 8분 시간 배분: 0~5분 개별. 5~7분 발표. 7~8분 액션.

BRIDGE

“1강 끝. 잠깐 쉬고 2강 ‘Hooks’로.”

INTRO · TITLE

Hooks — 정책 자동 적용

2강 — PreToolUse · PostToolUse 본격

🕒 3분 · 시작

SAY

“2강 ‘Hooks’입니다. 6편의 본격 핵심.”

SAY

“2강 60분 다섯 — (1) Hooks 풀이, (2) PreToolUse 사전 검증, (3) PostToolUse 사후 처리, (4) 추가 이벤트, (5) 작성 패턴 + 한국 사례 + 실습.”

SAY

“2강 약속 — Hooks는 ‘본격 코딩’. 단 ‘초보자 톤’ — 사내 IT가 작성하고 우리는 ‘어떤 Hooks 가치’ 결정.”

Q

도입 질문 1: “Hooks 들어본 분?” → 5편에서 본 분 있음. → ‘5편 개념 + 6편 본격.’

NOTE

★ 강사 핵심 배경지식 — Hooks의 의미: settings.json이 ‘정적 권한’이라면 Hooks는 ‘동적 동작’. ‘사용자 요청 → 도구 호출 → 응답’ 흐름 사이에 자동 함수 끼우기. 강력한 자동화.

NOTE

강사 배경지식 — 2강 60분 배분: 0~3분 인트로. 3~6분 학습목표. 6~14분 Hooks 풀이. 14~22분 PreToolUse. 22~30분 PostToolUse. 30~38분 추가 이벤트. 38~46분 작성 패턴. 46~52분 한국 사례. 52~58분 실습. 58~60분 마무리.

TIP

강사 함정 — Hooks 코드 깊이 X: ‘이벤트 + 가치’만.

BRIDGE

“학습 목표 빠르게.”

OBJECTIVES

60분 후, 이 3가지를 한다

- ① Hooks 이벤트 8가지+ 안다
- ② PreToolUse·PostToolUse 작성 패턴
- ③ ‘우리 회사 첫 Hook 설계서’ 완성

🕒 3분 · 학습목표

SAY

“60분 후 셋. Hooks 이벤트, 작성 패턴, 첫 Hook 설계서.”

SAY

“2강 핵심 메시지 — ‘첫 Hook 1개 작성 + 사고 방지 + 점진 확장’.”

NOTE

★ 강사 핵심 배경지식 — 2강 성공 기준: 청중이 ‘우리 회사 첫 Hook 후보 1개’ 결정. 본격 작성은 사내 IT.

TIP

강사 진행 팁 — 학습 목표 3분 안에.

BRIDGE

“본문 시작 — Hooks 풀이.”

WHAT IS HOOKS

Hooks — 동작 전후 자동 함수

■ 불변층 — 8가지+ 이벤트

- 수동 검증·정책 (Hooks 없이) — 직원이 ‘민감 데이터 송신 시도’ → 사후 발견 → 처벌. 시간 큼·사고 가능.
- 자동 검증·차단 (Hooks 사용) — PreToolUse Hook이 ‘민감 데이터 감지 → 차단 → 알림’. 사전 방지. 시간 0.

🕒 6분 · Hooks 풀이

SAY

“Hooks 풀이. Claude Code 동작 전후 자동 함수.”

SAY

“이벤트 8가지+ — (1) PreToolUse (도구 전). (2) PostToolUse (도구 후). (3) UserPromptSubmit (사용자 입력 시). (4) SessionStart (세션 시작). (5) SessionEnd. (6) Stop (정지). (7) Notification (알림). (8) PreCompact (압축 전). + 추가.”

SAY

“PreToolUse 예 — ‘민감 데이터 키워드 감지 → Edit 차단’. 사고 사전 방지.”

SAY

“PostToolUse 예 — ‘git commit 후 자동 슬랙 알림 + CI 트리거’. 사후 자동.”

SAY

“원칙 — 인간 검증 X 코드 검증. 직원 실수 X 시스템 차단.”

STORY

Hooks 공식 출처. Anthropic Claude Code docs.claude.com/claude-code/hooks. ‘8 events + JSON 입력 + 종료 코드 + JSON 응답’. 본 강좌 6편 2강 = 풀이 + 핵심 4 이벤트.

STORY

Hooks 동작 흐름 (강사 대응용). (1) 사용자 ‘이 파일 수정’ 요청. (2) Claude Code PreToolUse Hook 호출 — JSON으로 ‘무엇 할지’ 전달. (3) Hook 함수 (Bash·Python) 실행. (4) 종료 코드 0 = OK, 2 = 차단, 응답 JSON으로 추가 메시지. (5) 0이면 Edit 실행, 2면 차단.

STORY

Anthropic 사내 Hooks 사례 (강사 대응용). ‘How Anthropic teams use Claude Code’ 인용 — ‘every commit gets Hook-checked’. 사내 모든 변경에 Hook 자동 검증. ‘직원 실수 = 시스템 차단’ 운영.

NOTE

★ 강사 핵심 배경지식 — Hooks의 ‘4대 가치’: (1) 사고 방지 — 민감 데이터·위험 명령 사전. (2) 자동화 — 슬랙 알림·CI 트리거. (3) 정책 적용 — 회사 표준 자동. (4) 감사 로그 — 모든 동작 기록. 4대 결합.

NOTE

강사 배경지식 — Hooks 작성 언어: Bash 또는 Python 함수. settings.json에 명령 등록. ‘bash script.sh’ 또는 ‘python check.py’. 사내 IT 1주 학습.

NOTE

강사 배경지식 — Hooks ‘부담 함정’: Hook 함수 자체가 느림 → 매 작업 지연. ‘100ms+ 지연 X’ 원칙. 무거운 작업은 비동기 (PostToolUse 큐). 본 강좌 11편 자세히.

Q

예상 청중 질문 1: ‘우리 IT가 작성 가능?’ → 답: ‘Bash·Python 학습. 1~2주. 첫 Hook 1개 작성 → 점진 추가.’

Q

예상 청중 질문 2: ‘Hook 사고 시?’ → 답: ‘Hook 자체 버그 = 모든 작업 영향. 충분 테스트 + 작게 시작. log·debug 패턴.’

Q

예상 청중 질문 3: ‘Hooks 우회 가능?’ → 답: ‘Enterprise 강제 시 X. 사용자·프로젝트만이면 우회 가능 — Enterprise 권장.’

TIP

강사 함정 — Hooks 코드 깊이 X: ‘이벤트 + 가치’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 풀이. 1~2분 8이벤트. 2~3분 예. 3~4분 동작 흐름. 4~5분 부담 함정. 5~6분 청중 질문.

BRIDGE

“풀이 봤다면 — PreToolUse 디테일.”

PRETOOLUSE

PreToolUse — 사전 검증·차단

■ 불변층 — 안전의 핵심

- ★ 이벤트 — 도구 실행 직전
- ★ 입력 — Tool명 + 인자 (JSON)
- ★ 종료 코드 — 0 (OK) / 2 (차단) / 기타
- ★ 활용 1 — 민감 데이터 감지 + 차단
- ★ 활용 2 — 위험 명령 (rm·sudo) 차단
- ★ 활용 3 — 회사 정책 (커밋 메시지 형식) 검증

SAY

“PreToolUse — 사전 검증·차단. 안전 핵심.”

SAY

“이벤트 — 도구 실행 직전. Edit·Bash 등 모든 도구 호출 전.”

SAY

“입력 — Tool명 + 인자. JSON. ‘{tool: Edit, file: /home/user/secrets.txt}’ 같은.”

SAY

“종료 코드 — 0 = OK. 2 = 차단 + 에이전트에 ‘왜 차단’ 메시지. 기타 = 에러.”

SAY

“활용 1 민감 데이터. ‘이메일·전화·주민번호’ 정규식 매치 → 차단. 사내 PII 보호.”

SAY

“활용 2 위험 명령. ‘rm·sudo·삭제’ 명령 차단. settings.json deny 백업.”

SAY

“활용 3 정책. ‘git commit 메시지 형식 검증’. ‘feat·fix·docs’ 접두 강제.”

STORY

PreToolUse 입력 JSON 예시 (강사 대응용). { 'tool_name': 'Edit', 'tool_input': { 'file_path': '/home/user/.env', 'old_string': 'API_KEY=xxx', 'new_string': 'API_KEY=yyy' } }. Hook이 .env 감지 → 차단 + 메시지 '환경 파일은 사내 비밀 — Edit 차단'.

STORY

한국 회사 PreToolUse 사례 (강사 대응용). 일부 IT 회사 — (1) 주민번호 정규식 매치 차단. (2) rm·sudo 차단. (3) .env·secrets/ 폴더 차단. (4) git commit feat/fix/docs 접두 강제. 4종 기본.

STORY

PreToolUse 응답 JSON (강사 대응용). 종료 코드 0 + `{"continue": true, "systemMessage": "OK"}` JSON. 2 + `{"reason": "PII 감지"}` JSON. Claude Code가 에이전트에 ‘차단 이유’ 전달 → 에이전트 다른 접근 시도.

NOTE

★ 강사 핵심 배경지식 — PreToolUse ‘5단계 작성’: (1) 보호 대상 정의 (PII·secrets 등). (2) 감지 로직 (정규식·키워드·DB 조회). (3) 차단 또는 경고 결정. (4) 메시지 작성 (에이전트가 이해 가능). (5) 테스트. 1주 작성.

NOTE

강사 배경지식 — PreToolUse ‘성능 함정’: 매 작업 호출. ‘100ms 이상 지연’ 시 사용자 답답함. 무거운 검증은 ‘캐시 + 비동기’. 본 강좌 11편.

NOTE

강사 배경지식 — 정규식 라이브러리: (1) Microsoft Presidio — PII 다국어. (2) KoNLPy — 한국어. (3) 자체 정규식. 한국어 PII 강박이면 Presidio·KoNLPy.

Q

예상 청중 질문 1: ‘첫 PreToolUse 하나?’ → 답: “.env·secrets/ Edit 차단’ 가장 단순 + 가치. 1시간 작성.’

Q

예상 청중 질문 2: ‘에이전트 어떻게 반응?’ → 답: “차단 이유’ 메시지 → 다른 접근. 예 — ‘파일 직접 X .env.example 검토’ 자동.’

Q

예상 청중 질문 3: ‘false positive 우려?’ → 답: ‘있어요. 1주 운영 + 사용자 피드백 + 조정. 본 강좌 11편.’

TIP

강사 함정 — JSON 디테일 X: ‘구조 + 활용’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 이벤트. 1~3분 입력·종료. 3~4분 3활용. 4~5분 작성 5단계. 5~6분 청중 질문.

BRIDGE

“PreToolUse 봤다면 — PostToolUse.”

POSTTOOLUSE

PostToolUse — 사후 처리·자동화

■ 불변증 — 자동화 핵심

- ★ 이벤트 — 도구 실행 직후
- ★ 입력 — Tool명 + 인자 + 결과
- ★ 활용 1 — 슬랙 알림 (Edit·git commit 후)
- ★ 활용 2 — 자동 테스트 (Edit 후 npm test)
- ★ 활용 3 — 감사 로그 (모든 변경 DB 기록)
- ★ 활용 4 — CI 트리거 (git push 후 자동 빌드)

🕒 6분 · PostToolUse

SAY

“PostToolUse — 사후 처리·자동화.”

SAY

“이벤트 — 도구 실행 직후.”

SAY

“입력 — Tool명 + 인자 + 결과. ‘파일 수정 결과’, ‘명령 출력’ 등.”

SAY

“활용 1 슬랙 알림. ‘Edit·git commit 후 슬랙 #ai-changes 자동 알림’. 팀 인지.”

SAY

“활용 2 자동 테스트. ‘Edit 후 npm test 자동’. 사고 즉시 발견.”

SAY

“활용 3 감사 로그. ‘모든 변경 사내 DB 기록’. 누가·언제·무엇. 본 강좌 9편.”

SAY

“활용 4 CI 트리거. ‘git push 후 자동 빌드·배포’. 4편 MCP 결합.”

STORY

PostToolUse 입력 JSON 예시 (강사 대응용). { 'tool_name': 'Edit', 'tool_input': {...}, 'tool_result': { 'success': true, 'file_path': '...', 'changes': '...' } }. Hook이 ‘파일 수정 성공’ 확인 → 슬랙 알림 + 테스트 + 로그.

STORY

한국 회사 PostToolUse 사례 (강사 대응용). (1) git commit 후 슬랙. (2) src/ 수정 후 ESLint. (3) DB 변경 후 백업. (4) 배포 후 모니터링 알림. 4가지 표준.

STORY

PostToolUse 비동기 패턴 (강사 대응용). Hook 자체 빠르게 종료 + 무거운 작업 백그라운드. 예 — Hook에서 ‘nohup script.sh &’ → 즉시 종료 + 백그라운드 슬랙. 사용자 답답함 X.

NOTE

★ 강사 핵심 배경지식 — PostToolUse ‘5가지 권장’: (1) 슬랙 알림 (중요 변경만). (2) 자동 테스트 (코드 변경 시). (3) 감사 로그 (모든 변경). (4) CI 트리거 (push 후). (5) 비동기 큐. 5가지 단계별 도입.

NOTE

강사 배경지식 — 슬랙 알림 ‘부담 함정’: 모든 변경 알림 → 슬랙 시끄러움. ‘중요 변경만 + 시간대 제한 + 채널 분리’. 본 강좌 11편.

Q

예상 청중 질문 1: ‘첫 PostToolUse 하나?’ → 답: “git commit 후 슬랙 알림’ 가장 단순. 1시간 작성.’

Q

예상 청중 질문 2: ‘성능 영향?’ → 답: ‘비동기 X 동기면 영향. 비동기 패턴 권장. 본 강좌 11편.’

Q

예상 청중 질문 3: ‘모든 변경 로그 DB?’ → 답: ‘권장. ELK Stack 또는 사내 DB. 본 강좌 9편 자세히.’

TIP

강사 함정 — Hook 코드 깊이 X: ‘활용 + 가치’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 이벤트. 1~3분 4활용. 3~4분 비동기. 4~5분 부담 함정. 5~6분 청중 질문.

BRIDGE

“PostToolUse 봤다면 — 추가 이벤트.”

OTHER EVENTS

추가 이벤트 — UserPromptSubmit · Stop · 기타

■ 불변증 — 6가지+ 추가 이벤트

- ★ UserPromptSubmit — 사용자 입력 시
- ★ SessionStart·SessionEnd — 세션 경계
- ★ Stop — 에이전트 정지 시
- ★ Notification — 사용자 알림 시
- ★ PreCompact — 컨텍스트 압축 전
- ★ 각 이벤트 — 특정 패턴 자동화

🕒 5분 · 추가 이벤트

SAY

“추가 이벤트 6가지+.”

SAY

“UserPromptSubmit — 사용자 입력 시. ‘민감 키워드 검증’ 또는 ‘회사 정책 안내 메시지 추가’.”

SAY

“SessionStart — 세션 시작 시. ‘환영 메시지’, ‘회사 정책 안내’.”

SAY

“SessionEnd — 세션 끝 시. ‘세션 요약 저장’, ‘ROI 데이터 기록’.”

SAY

“Stop — 에이전트 정지 시. ‘완료 알림’ 또는 ‘다음 단계 제안’.”

SAY

“Notification — 사용자 알림 시. ‘슬랙 미러링’.”

SAY

“PreCompact — 컨텍스트 압축 전. ‘중요 정보 저장’.”

STORY

UserPromptSubmit 활용 (강사 대응용). 사용자 ‘우리 회사 매출 정보 알려’ → Hook이 ‘민감 정보 — 임원만’ 인지 → 차단 또는 ‘제한적 답 가능’ 추가. 정책 사전 적용.

STORY

SessionStart 활용 (강사 대응용). 매 세션 시작 시 — ‘오늘 우리 회사 정책 변경’, ‘현재 사용량’, ‘권장 작업’ 안내. 직원 인지 + 사용 패턴.

NOTE

★ 강사 핵심 배경지식 — 추가 이벤트의 ‘가치’: 핵심 PreToolUse·PostToolUse 외에 ‘세션·입력 흐름’ 추가 자동화. ‘회사 전체 정책 사이클’ 코드화.

Q

예상 청중 질문 1: ‘추가 이벤트 다 도입?’ → 답: ‘X. PreToolUse + PostToolUse 우선. 추가는 ‘6개월 운영 후 필요’ 시 도입.’

Q

예상 청중 질문 2: ‘UserPromptSubmit 검열 X?’ → 답: ‘회사 정책 안내 + 위험 키워드 차단. 검열보다 ‘안전 안내’.’

TIP

강사 함정 — 추가 이벤트 디테일 X: ‘목록 + 가치’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 6이벤트. 1~3분 각 30초. 3~4분 활용 예. 4~5분 청중 질문.

BRIDGE

“이벤트 봤다면 — 작성 패턴.”

WRITE PATTERNS

Hook 작성 패턴 — Bash·Python

■ 불변층 — 4단계 작성

- ★ 1단계 — 이벤트 + 활용 결정
- ★ 2단계 — settings.json에 hook 등록
- ★ 3단계 — Bash·Python 함수 작성
- ★ 4단계 — 테스트 + 점진 운영
- ★ 패턴 — 단순 Bash + 복잡 Python
- ★ 디버깅 — log + dry run

🕒 5분 · 작성 패턴

SAY

“Hook 작성 4단계.”

SAY

“1단계 결정. 어떤 이벤트 + 어떤 활용. ‘.env Edit 차단’.”

SAY

“2단계 settings.json 등록. hooks 키에 ‘PreToolUse: [{matcher: Edit, hooks: [{type: command, command: bash script.sh}]]’ 같은.”

SAY

“3단계 함수 작성. 단순 — Bash 1줄. 복잡 — Python 1파일. ‘JSON 입력 → 검증 → 종료 코드 + JSON 응답’.”

SAY

“4단계 테스트 + 점진. ‘1주 dry run + 사용자 피드백 + 본격’.”

SAY

“패턴 — 단순 단축은 Bash. 복잡 검증은 Python. 사내 IT 능력에 따라.”

SAY

“디버깅 — log + dry run. ‘echo 로 입력 확인’ + ‘실제 차단 X 경고만’.”

STORY

Hook Bash 예시 (강사 대응용). `#!/bin/bash. input=$(cat). file=$(echo $input | jq -r .tool_input.file_path). if [[ $file == *.env ]]; then echo "{\"reason\": \".env Edit 차단\"}" >&2; exit 2; fi. exit 0.` 약 5줄. 단순.

STORY

Hook Python 예시 (강사 대응용). `import sys, json, re. input = json.loads(sys.stdin.read()). file = input['tool_input'].get('file_path', ''). if re.search(r'\\.env|secrets/', file): json.dump({'reason': '민감 파일'}, sys.stderr); sys.exit(2).`

NOTE

★ 강사 핵심 배경지식 — Hook 작성 ‘함정 5가지’: (1) 무한 루프 — Hook이 다른 Hook 트리거. (2) 성능 — 100ms+ 지연. (3) false positive — 정상 차단. (4) false negative — 사고 통과. (5) 디버깅 어려움. 5가지 다 대응.

NOTE

강사 배경지식 — Hook ‘권장 도구’: jq (JSON 파싱), Microsoft Presidio (PII), KoNLPy (한국어), grep·sed (텍스트). Bash + 4도구 = 대부분 Hook 가능.

Q

예상 청중 질문 1: ‘우리 IT 작성 가능?’ → 답: ‘Bash·Python 학습. 1~2주. 첫 Hook 1주.’

Q

예상 청중 질문 2: ‘외부 도구 호출?’ → 답: ‘가능. curl·API 호출. 단 ‘성능 영향’ — 비동기 패턴.’

Q

예상 청중 질문 3: ‘Hook 라이브러리?’ → 답: ‘GitHub ‘claude-code hooks’ 검색. 사내 공유 패키지도 가능.’

TIP

강사 함정 — 코드 깊이 X: ‘패턴’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 4단계. 1~3분 작성 예. 3~4분 함정·디버깅. 4~5분 청중 질문.

BRIDGE

“작성 봤다면 — 한국 사례 + 실습.”

KOREA HOOKS

한국 회사 Hooks 사례 5종

■ 가변층 — 2026.6 사례

- ★ 1 — 민감 파일 (.env·secrets/) Edit 차단
- ★ 2 — 한국 PII (이름·주민·전화) 정규식 차단
- ★ 3 — git commit 메시지 형식 강제 (feat·fix·docs)
- ★ 4 — 코드 수정 후 자동 ESLint·테스트
- ★ 5 — git push 후 슬랙 #ai-changes 알림
- ★ 5종 — 1개월 안 도입 권장

🕒 5분 · 한국 사례

SAY

“한국 회사 Hooks 사례 5종. 1개월 안 도입 권장.”

SAY

“1 — 민감 파일 차단. .env·secrets/ Edit 차단. PreToolUse. 1시간.”

SAY

“2 — 한국 PII 차단. 이름·주민·전화 정규식. KoNLPy 또는 Presidio. 1일.”

SAY

“3 — git commit 메시지 강제. feat·fix·docs·refactor 접두. PreToolUse Bash. 1일.”

SAY

“4 — 코드 후 자동 ESLint·테스트. PostToolUse. CI 결합. 1주.”

SAY

“5 — git push 후 슬랙 알림. PostToolUse + 슬랙 webhook. 2시간.”

SAY

“5종 1개월 도입 = 본격 안전 운영.”

STORY

한국 회사 Hooks 비용·시간 (강사 대응용). 5종 작성 — 사내 IT 1명 × 1개월. 비용 0 (사내 인력) 또는 외주 200~500만 원. ROI — 사고 방지 + 효율 자동. 1년 회수.

NOTE

★ 강사 핵심 배경지식 — 5종 우선순위: (1) 1·2 (보안) — 1주 안. (2) 3 (정책) — 2주. (3) 4 (품질) — 3주. (4) 5 (협업) — 1개월. 1개월 안 5종 완성.

Q

예상 청중 질문 1: ‘5종 다 도입?’ → 답: ‘권장. 1개월 작업. 사고 방지 + 효율.’

Q

예상 청중 질문 2: ‘우리 IT 1명 1개월 부담?’ → 답: ‘외주 옵션. 200~500만 원. 사내 학습 가치 큼 → 사내 권장.’

Q

예상 청중 질문 3: ‘다른 Hooks?’ → 답: ‘많음. 5종 후 ‘사내 발견’에 따라 추가.’

TIP

강사 함정 — ‘완벽 5종’ 압박 X: ‘우선 1·2 시작’.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 5종. 1~3분 각 30초. 3~4분 우선순위. 4~5분 청중 질문.

BRIDGE

“사례 봤다면 — 실습.”

WORKSHOP 2

실습 — 우리 회사 첫 Hook 설계서

■ 적용 (워크시트)

- Q1 · 첫 Hook 후보 — 5종 중 무엇?
- Q2 · 이벤트 — PreToolUse / PostToolUse?
- Q3 · 함수 — Bash 또는 Python?
- Q4 · 작성자 — 사내 IT 또는 외주?
- Q5 · 일정 — 1주 / 2주 / 1개월?
- ★ 다음 액션 — IT 1명 1주 안 첫 Hook + 테스트

🕒 8분 · 실습

SAY

“2강 실습 — 첫 Hook 설계서 5칸.”

DO

조별 실습 (5분, 3~4인 1조).

SAY

“권장 시작 — 민감 파일 차단 + PreToolUse + Bash + 사내 IT + 1주. 가장 안전 + 간단.”

Q

발표 요청: “첫 Hook 후보?” → 보통 ‘민감 파일’.

NOTE

★ 강사 핵심 배경지식 — 결정서 5칸 가이드: (1) Q1 — 1·2 우선. (2) Q2 — Pre가 더 안전. (3) Q3 — Bash 단순. (4) Q4 — 사내 학습. (5) Q5 — 1주 시작.

NOTE

강사 배경지식 — 다음 주 액션 3가지: (1) IT 1명 첫 Hook 1주. (2) 1주 dry run 테스트. (3) git 공유 + 사내 표준.

TIP

강사 함정 — ‘완벽 결정 압박’ X.

TIP

강사 진행 팁 — 8분 시간 배분: 0~5분 개별. 5~7분 발표. 7~8분 액션.

BRIDGE

“2강 끝. 잠깐 쉬고 3강 ‘Skills·Slash·MCP’로.”

WRAP

2강 정리 — Hooks 핵심

■ 마무리

- Hooks = 동작 전후 자동 함수
- ★ PreToolUse — 사전 검증·차단 (안전)
- ★ PostToolUse — 사후 자동·알림 (자동화)
- ★ 한국 5종 — 1개월 안 도입 권장
- ★ 다음 — 3강 Skills·Slash·MCP

🕒 3분 · 정리

SAY

“2강 정리. Hooks = 동작 전후 자동. PreToolUse 안전 + PostToolUse 자동화.”

SAY

“핵심 메시지 — ‘첫 Hook 1개 + 1개월 5종 + 사내 표준’.”

SAY

“3강 예고 — Skills (도메인 지식) + Slash (단축) + MCP (외부) 결합. 하네스 풀스택 완성.”

TIP

강사 진행 팁 — 3분 안에.

BRIDGE

“쉬는 시간 10분. 3강에서.”

INTRO · TITLE

Skills · Slash · MCP

3강 — 부서 도메인 지식 패키징 + 단축 + 외부 통합

🕒 3분 · 시작

SAY

“3강 ‘Skills + Slash + MCP’입니다. 6편 마지막. 하네스 풀스택 완성.”

SAY

“3강 60분 다섯 — (1) Skills 풀이, (2) Skills 작성, (3) Slash Commands, (4) MCP 통합, (5) 풀스택 + 사례 + 실습.”

Q

도입 질문 1: “‘Skills’ 5편에서 본 분?” → 보통 다수. → ‘좋아요. 6편 본격.’

NOTE

★ 강사 핵심 배경지식 — Skills의 위치: settings.json·Hooks가 ‘권한·정책’이라면 Skills는 ‘지식 패키징’. ‘우리 회사 어떻게 일하는가’ 코드화.

NOTE

강사 배경지식 — 3강 60분 배분: 0~3분 인트로. 3~6분 학습목표. 6~14분 Skills 풀이. 14~22분 Skills 작성. 22~30분 Slash Commands. 30~38분 MCP 통합. 38~46분 풀스택. 46~52분 한국 사례. 52~58분 실습. 58~60분 마무리.

TIP

강사 함정 — Skills 코드 깊이 X: ‘구조 + 작성 패턴’만.

BRIDGE

“학습 목표 빠르게.”

OBJECTIVES

60분 후, 이 3가지를 한다

- ① Skills + Slash + MCP 결합 안다
- ② 우리 회사 도메인 Skills 1개 설계
- ③ ‘하네스 폴스택 1페이지’ 완성

🕒 3분 · 학습목표

SAY

“60분 후 셋. 결합, 도메인 Skills, 하네스 폴스택.”

SAY

“3강 핵심 메시지 — ‘우리 회사 지식 1개 → Skills 패키지 → 전사 공유’.”

NOTE

★ 강사 핵심 배경지식 — 3강 성공 기준: 청중이 ‘우리 회사 도메인 Skills 1개 후보’ 결정.

TIP

강사 진행 팁 — 학습 목표 3분 안에.

BRIDGE

“본문 시작 — Skills 풀이.”

SKILLS WHAT

Skills — 도메인 지식 패키징

■ 불변층 — Anthropic 2025 신규

- ★ Skills — ‘우리 회사 어떻게 일하는가’ 코드 패키지
- ★ 구조 — 폴더 + SKILL.md + 코드·예
- ★ 예 — ‘영업 보고서 작성 스킬’, ‘법무 계약서 검토’
- ★ 위치 — .claude/skills/<name>/SKILL.md
- ★ 자동 발견 — Claude Code가 ‘어떤 작업’ 인식 → 자동 호출
- ★ 사내 공유 — git 커밋 + 전사 동기화

🕒 6분 · Skills 풀이

SAY

“Skills — 도메인 지식 패키징. Anthropic 2025 신규.”

SAY

“‘우리 회사 어떻게 일하는가’ 코드 패키지. 영업 보고서·법무 계약서·사내 코드 스타일 등.”

SAY

“구조 — 폴더 + SKILL.md (마크다운 설명) + 코드·예시. ‘.claude/skills/<name>/SKILL.md’.”

SAY

“예 — 영업 보고서 작성 스킬. SKILL.md에 ‘회사 양식 + 톤 + 데이터 출처’ 설명. 예시 보고서 3건.”

SAY

“자동 발견 — Claude Code가 ‘어떤 작업’ 인식 → 자동 호출. ‘영업 보고서 작성’ 입력 → Skills 자동 활성화.”

SAY

“사내 공유 — git 커밋 + 전사 동기화. 영업팀 1명 작성 → 전 영업팀 공유.”

STORY

Skills 출처. Anthropic 'Introducing Agent Skills' 2025. URL: claude.com/blog/skills. 'structured domain knowledge packaging'. Claude Code·Claude Desktop 호환.

STORY

Skills 폴더 구조 (강사 대응용). `.claude/skills/sales-report/SKILL.md` + `.claude/skills/sales-report/examples/`. SKILL.md에 '회사 양식·톤·데이터·작성 흐름'. examples에 '과거 보고서 3건'.

STORY

한국 회사 Skills 사례 (강사 대응용). 일부 회사 — '사내 코드 스타일 가이드', 'API 문서 작성', '회의록 양식', '영업 보고서' 등 10~20 스킬. 1년 운영 후 사내 표준.

NOTE

★ 강사 핵심 배경지식 — Skills의 '5가지 가치': (1) 1회 작성 + 전사 공유. (2) 자동 호출 — 사용자 명시 X. (3) 도메인 전문성 — 신입도 시니어 수준. (4) 표준화 — 부서별 일관. (5) git history — 정책 변경 추적.

NOTE

강사 배경지식 — Skills vs CLAUDE.md 차이: CLAUDE.md = 프로젝트 전체 가이드 (1개). Skills = 작업별 패키지 (다수). CLAUDE.md에 '일반 가이드', Skills에 '특수 작업'. 둘 다 사용.

NOTE

강사 배경지식 — Skills '자동 호출' 메커니즘: SKILL.md 'description' 키워드 + 작업 매칭. '영업 보고서 작성' 사용자 입력 → Claude Code가 'sales-report' Skill description 검색 → 매칭 → 자동 활성화.

Q

예상 청중 질문 1: '우리 회사 Skills 후보?' → 답: '자주 반복 + 표준 양식 있는 작업. 영업 보고서·회의록·코드 리뷰 등.'

Q

예상 청중 질문 2: '비IT 부서 Skills 작성?' → 답: '마크다운 작성 OK. 단 자동화 코드 = IT 협업. 영업팀 '양식 + 예시' 작성 + IT 결합.'

Q

예상 청중 질문 3: ‘Skills 몇 개부터?’ → 답: ‘첫 도입 — 1~3개. 6개월 후 5~10. 1년 후 20+. 점진 확장.’

TIP

강사 함정 — Skills 코드 깊이 X: ‘구조 + 가치’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 풀이. 1~3분 구조·예. 3~4분 자동 발견. 4~5분 vs CLAUDE.md. 5~6분 청중 질문.

BRIDGE

“풀이 봤다면 — 작성 패턴.”

SKILLS WRITE

Skills 작성 패턴 — 5단계

■ 불변증 — 사내 표준 작성 흐름

- ★ 1단계 — 후보 선정 (자주 반복 + 표준 양식)
- ★ 2단계 — SKILL.md 작성 (description·tools·instructions)
- ★ 3단계 — examples 폴더 (과거 결과 3~5건)
- ★ 4단계 — 자동 호출 테스트 (사용자 입력 → 매칭)
- ★ 5단계 — git 공유 + 사용자 피드백 + 개선
- ★ 시간 — 첫 Skill 1주, 이후 2~3일

🕒 6분 · Skills 작성

SAY

“Skills 작성 5단계. 첫 Skill 1주.”

SAY

“1단계 후보. 자주 반복 + 표준 양식 작업. 영업 보고서·회의록·코드 리뷰 등.”

SAY

“2단계 SKILL.md. description (작업 설명) + tools (필요 도구) + instructions (작성 흐름). 마크다운.”

SAY

“3단계 examples. 과거 결과 3~5건. ‘이런 입력 → 이런 결과’ 패턴.”

SAY

“4단계 테스트. 사용자 입력 → Skills 자동 호출 확인. ‘영업 보고서 작성’ 매칭.”

SAY

“5단계 git 공유 + 피드백 + 개선. 1주 운영 + 사용자 피드백 + 업데이트.”

STORY

SKILL.md 예시 구조 (강사 대응용). YAML frontmatter — name·description·tools·model. 본문 — ‘작업 흐름’ 마크다운. ‘1. 자료 수집 → 2. 초안 → 3. 회사 양식 적용 → 4. 검토’ 같은.

STORY

한국 회사 Skills 작성 시간 (강사 대응용). 첫 Skill — 1주 (학습 + 작성 + 테스트). 이후 — 2~3일. 1개월 5~10 Skills 가능. 사내 IT + 부서 챔피언 협업.

NOTE

★ 강사 핵심 배경지식 — Skills 작성의 ‘함정’: (1) ‘너무 일반’ — 매칭 X. (2) ‘너무 특수’ — 사용 X. (3) examples 부족 — 품질 ↓. (4) git 공유 X — 개인 사용. (5) 피드백 X — 개선 X. 5가지 함정 피하기.

NOTE

강사 배경지식 — SKILL.md ‘description’ 작성: 명확한 ‘언제 호출’ 설명. ‘이 스킬은 영업 보고서를 작성할 때 사용. 회사 양식·톤·데이터’. Claude가 자동 인식할 수 있게.

Q

예상 청중 질문 1: ‘우리 회사 첫 Skill 후보?’ → 답: ‘일반 — 회의록 양식. IT — 사내 코드 스타일. 영업 — 보고서. 부서별 가장 자주 + 표준.’

Q

예상 청중 질문 2: ‘비IT 부서 Skills 가능?’ → 답: ‘마크다운 + 예시 = OK. IT 결합 X도 가능. 자동화 코드 = IT.’

Q

예상 청중 질문 3: ‘Skills 사용 확인?’ → 답: ‘세션 끝 ‘Skills 사용 횟수’ 로그 + 사용자 피드백.’

TIP

강사 함정 — 코드 깊이 X: ‘5단계 + 함정’만.

TIP

강사 진행 팁 — 6분 시간 배분: 0~1분 5단계. 1~3분 SKILL.md 구조. 3~4분 함정 5가지. 4~5분 시간. 5~6분 청중 질문.

BRIDGE

“작성 봤다면 — *Slash Commands.*”

SLASH

Slash Commands — 단축 명령

■ 불변증 — 자주 쓰는 명령 단축

- ★ Slash Commands — ‘/단어’ 단축 명령
- ★ 위치 — .claude/commands/<name>.md
- ★ 예 — /report (보고서), /review (PR 리뷰), /deploy (배포)
- ★ 구조 — 마크다운 + 명령 흐름
- ★ 차이 — Skills (자동 호출) vs Slash (사용자 명시)
- ★ git 공유 — 프로젝트 레벨

🕒 5분 · Slash

SAY

“Slash Commands — ‘/단어’ 단축 명령.”

SAY

“위치 — .claude/commands/<name>.md 파일.”

SAY

“예 — /report (보고서 작성), /review (PR 리뷰), /deploy (배포), /test (테스트).”

SAY

“구조 — 마크다운 + 명령 흐름. ‘1. 자료 수집 → 2. 분석 → 3. 답’ 같은.”

SAY

“차이 — Skills는 ‘자동 호출’, Slash는 ‘사용자 명시’. Skills가 자연, Slash가 단축.”

SAY

“git 공유 — 프로젝트 레벨 + 팀 동기화.”

STORY

Slash Commands 출처. Claude Code docs.claude.com/claude-code/slash-commands.
‘.claude/commands/<name>.md’ 자동 등록. 사용 ‘/name’.”

STORY

한국 회사 Slash 사례 (강사 대응용). /commit (commit 메시지 생성), /test (테스트 작성), /review (코드 리뷰), /docs (문서화). 4~10 commands 표준.

NOTE

★ 강사 핵심 배경지식 — Slash vs Skills 결정: 자주 사용 + 사용자가 ‘명시 가능’ — Slash. 다양한 입력에 ‘자동 호출’ — Skills. 둘 다 사용 권장.

Q

예상 청중 질문 1: ‘우리 회사 첫 Slash?’ → 답: ‘/commit (commit 메시지 형식 자동) 가장 단순 + 가치.’

Q

예상 청중 질문 2: ‘Slash + Skills 결합?’ → 답: ‘가능. ‘/report’ Slash가 Sales Report Skills 호출. 결합 자연.’

TIP

강사 함정 — Slash 디테일 깊이 X: ‘위치 + 가치’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 풀이. 1~2분 예. 2~3분 vs Skills. 3~4분 한국 사례. 4~5분 청중 질문.

BRIDGE

“Slash 봤다면 — MCP 통합.”

MCP INTEGRATION

MCP 통합 — 4편 + 6편 결합

■ 불변증 — 외부 시스템 + 하네스

- ★ MCP = 4편 3강. 외부 시스템 연결
- ★ Claude Code MCP 통합 — settings.json에 등록
- ★ Hooks + MCP — MCP 호출 전후 검증·로그
- ★ Skills + MCP — Skills 안에서 MCP 도구 호출
- ★ Slash + MCP — Slash가 MCP 자동 호출
- ★ 풀스택 — 4편 MCP + 6편 하네스 = 안전 자동

🕒 5분 · MCP 통합

SAY

“MCP 통합. 4편 3강 + 6편 결합.”

SAY

“MCP — 외부 시스템 연결 (4편 3강). Claude Code에 자동 통합.”

SAY

“settings.json에 MCP 서버 등록. ‘mcpServers’ 키 + URL + 인증.”

SAY

“Hooks + MCP — MCP 호출 전후 검증·로그. ‘ERP 매출 조회 후 슬랙 알림’.”

SAY

“Skills + MCP — Skills 안에서 MCP 도구 사용. ‘영업 보고서 스킬’이 ‘ERP MCP 매출 호출 → 보고서’.”

SAY

“Slash + MCP — ‘/sales-report’ Slash → MCP 호출 → 답.”

SAY

“풀스택 — 4편 MCP + 6편 하네스 = 안전 자동. ‘MCP 호출 + Hooks 검증 + Skills 흐름 + Slash 단축’ 결합.”

STORY

settings.json MCP 등록 예시 (강사 대응용). { 'mcpServers': { 'sales-erp': { 'command': 'node', 'args': ['./mcp-sales.js'] }, 'gmail': { 'url': 'https://gmail-mcp.local' } } }. 사내 + 외부 결합.

STORY

MCP 통합 한국 사례 (강사 대응용). 일부 회사 — Microsoft 365 MCP + Sales ERP MCP + Hooks 검증 + Skills 흐름. ‘직원 ‘이번 분기 매출 보고서’ → 자동 ERP 조회 + 보고서 작성 + 슬랙 검토 요청’. 풀스택 사례.

NOTE

★ 강사 핵심 배경지식 — 풀스택의 ‘5가지 가치’: (1) 외부 시스템 자동. (2) 정책 자동 적용. (3) 도메인 지식 적용. (4) 단축 명령. (5) 감사 로그. 5가지 결합 = ‘완전한 사내 AI’.

NOTE

강사 배경지식 — 풀스택 구축 순서: (1) MCP 서버 (4편). (2) settings.json + Hooks (6편 1·2강). (3) Skills (6편 3강 전반). (4) Slash + 결합 (6편 3강 후반). 6편 끝 = 풀스택 완성.

Q

예상 청중 질문 1: ‘풀스택 시간?’ → 답: ‘1~3개월 완성. 4편 + 5편 + 6편 단계별.’

Q

예상 청중 질문 2: ‘우리 회사 풀스택 가능?’ → 답: ‘IT 시니어 + 1~3개월. 또는 외주 500~2,000만 원.’

TIP

강사 함정 — 통합 코드 깊이 X: ‘결합 + 가치’만.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 통합. 1~3분 4결합. 3~4분 풀스택. 4~5분 청중 질문.

BRIDGE

“MCP 통합 봤다면 — 풀스택 + 한국 사례.”

FULLSTACK

하네스 풀스택 — 4요소 결합

■ 불변층 — 6편의 결론

- settings.json 정적 권한 — allow·ask·deny + env·model. 사내 표준 1회 작성 + 영구. 1주.
- Hooks 동적 자동 — Pre·Post·Session·기타 8이벤트+. 정책 자동 + 사고 방지. 1개월 5종.
- Skills 도메인 지식 — ‘우리 회사 어떻게’ 패키지. 자동 호출. 1년 20+ Skills.
- Slash + MCP 단축 + 외부 통합 — /단어 단축 + 외부 시스템. 풀스택 완성. 1~3개월.

🕒 5분 · 풀스택

SAY

“하네스 풀스택. 4요소 결합. 6편의 결론.”

SAY

“settings.json — 정적 권한. 1주.”

SAY

“Hooks — 동적 자동. 1개월 5종.”

SAY

“Skills — 도메인 지식. 1년 20+.”

SAY

“Slash + MCP — 단축 + 외부. 1~3개월 풀스택 완성.”

SAY

“순서 — settings (1주) → Hooks 1·2 (2주) → 첫 Skill (1주) → MCP 통합 (1개월) → 풀스택 (3개월).”

STORY

Anthropic 사내 풀스택 (강사 대응용). 'How Anthropic teams use Claude Code' — ‘settings + hooks + skills + commands + MCP’ 결합. 사내 전 직원 동일 환경. 한국 회사 참고 패턴.

NOTE

★ 강사 핵심 배경지식 — 풀스택의 ‘회사 효율’: 도입 전 — 정책 X 직원 실수 + 수동 작업 + 도메인 다양. 도입 후 — 정책 자동 + 도메인 표준 + 외부 통합. 효율 +50%+ + 사고 0.

Q

예상 청중 질문 1: ‘풀스택 완성 일정?’ → 답: ‘3개월 본격. 1년 후 사내 표준 정착.’

Q

예상 청중 질문 2: ‘우리 작은 회사 풀스택 X?’ → 답: ‘작은 회사도 풀스택 가능. 단 ‘Skills 5개’ 같은 작은 규모. 풀스택 비례.’

TIP

강사 함정 — ‘완벽 풀스택 압박’ X: ‘단계별 + 점진 확장’.

TIP

강사 진행 팁 — 5분 시간 배분: 0~1분 4요소. 1~3분 순서. 3~4분 가치. 4~5분 청중 질문.

BRIDGE

“풀스택 봤다면 — 실습.”

WORKSHOP 3

실습 — 하네스 풀스택 결정서

■ 적용 (워크시트)

- Q1 · 1주 — settings.json 표준?
- Q2 · 1개월 — Hooks 5종?
- Q3 · 3개월 — Skills 5개?
- Q4 · 1~3개월 — MCP 통합?
- Q5 · 6개월 — 풀스택 완성?
- ★ 다음 액션 — 이번 주 settings.json 시작

🕒 8분 · 실습

SAY

“3강 실습 — 하네스 풀스택 결정서 5칸.”

DO

조별 실습 (5분, 3~4인 1조).

SAY

“권장 — 1주 settings + 1개월 Hooks 5종 + 3개월 Skills 5개 + 3개월 MCP + 6개월 풀스택. 단계별.”

Q

발표 요청: “6개월 후 풀스택 가능?” → 다양.

NOTE

★ 강사 핵심 배경지식 — 결정서 5칸 가이드: (1) Q1 — 시니어 1명 1주. (2) Q2 — 1개월 단계별. (3) Q3 — 부서 협업. (4) Q4 — 4편 진행 시. (5) Q5 — 6개월 본격.

NOTE

강사 배경지식 — 다음 주 액션 3가지: (1) 시니어 1명 settings.json 1주 작성. (2) 첫 Hook (.env 차단) 1주. (3) 사내 표준 git 공유. 2주 안.

TIP

강사 함정 — ‘완벽 결정 압박’ X.

TIP

강사 진행 팁 — 8분 시간 배분: 0~5분 개별. 5~7분 발표. 7~8분 액션.

BRIDGE

“3강 + 6편 마무리.”

PART 6 WRAP

6편 정리 — 하네스 풀스택

■ 마무리 + 7편 예고

- 1강 — settings.json (권한 + Permission Modes)
- 2강 — Hooks (PreToolUse·PostToolUse + 5종)
- 3강 — Skills + Slash + MCP 결합
- ★ 1~6편 = ‘에이전트 안전 운영 16장 마스터’
- ★ 다음 — 7편(사내 에이전트 배포) 본격 전사 보급

🕒 5분 · 마무리

SAY

“6편 마무리. settings.json + Hooks + Skills + Slash + MCP = 하네스 풀스택.”

SAY

“추천 시작 — 시니어 1명 + 1주 settings + 1개월 Hooks 5종 + 3개월 Skills 5개 + 6개월 풀스택.”

SAY

“7편 예고 — ‘사내 에이전트 배포’. 6편이 ‘개발팀 하네스’였다면 7편은 ‘전사 보급’. 6개월 framework + Managed Agents + 부서별 도입.”

NOTE

★ 강사 핵심 배경지식 — 6편 끝 톤: ‘개발팀 + 하네스 → 사내 전사’ 진화. 7편이 ‘다음 단계’.

TIP

강사 진행 팁 — 5분 시간 배분.

BRIDGE

“6편 끝. 7편 ‘사내 에이전트 배포’에서.”

THANKS

6편 끝 — 7편에서

‘사내 에이전트 배포’ 본격 전사

🕒 2분 · 작별

SAY

“6편 마지막. 1·2·3강 180분 고생하셨습니다.”

SAY

“오늘 — settings + Hooks + Skills + Slash + MCP 하네스 풀스택. 회사 정책 코드화.”

SAY

“다음 — 7편 ‘사내 에이전트 배포’. 본격 전사 보급.”

SAY

“후기·질문 — visionc.co.kr.”

NOTE

마지막.

TIP

2분.

BRIDGE

“감사합니다.”